

SPEC — CRM (Admin customer console)

SECTION 06-operations-it

TYPE spec

STATUS

draft

OWNER Product

UPDATED 2026-06-16

Status: Draft · **Owner:** Product · **Last updated:** 2026-05

An internal, staff-only console for Fly GACA to see and manage the people and organizations on the platform — pilot users, their subscriptions, support threads, and the B2B pipeline of flight schools — in one place, instead of across the Firebase console, Stripe dashboard and an inbox.

Internal & operational. This console is for Fly GACA staff only. It reads customer data the platform already holds (accounts, the `entitlement` plan, billing status) under our Privacy Notice and PDPL obligations. It is **not** part of the shipped site, is `noindex`, and lives behind the existing staff-only `admin` custom claim (the one `functions/school.js` already enforces; see §5). It never exposes another user's data to a non-staff caller.

1. Why

Today, answering "who is this customer, what plan are they on, and what have they asked us?" means cross-referencing three tools by hand:

- **Firebase console** for the account (`users/{uid}` , which carries the `entitlement` plan field).
- **Stripe dashboard** (and RevenueCat, for mobile) for the subscription.
- **Email** (`i@flygaca.com`) for whatever they wrote in.

As Phase 5 (Money & flight schools) turns on paid plans and brings in organizations that buy seats, this gets worse fast. We need a single read-mostly surface that joins **account** → **entitlement** → **billing** → **support** → **pipeline**, so support is faster, churn is visible, and B2B leads don't fall through the cracks. The CRM is the operator's view of the customer; it reuses the data the rest of the product already generates rather than inventing a new source of truth.

2. Who

- **Staff (admin)** — Fly GACA team. An `admin` custom claim **already exists**: `functions/school.js` enforces it via `assertAdmin(req)` (`req.auth.token.admin === true`), set with `admin.auth().setCustomUserClaims(uid, { admin: true })`. The CRM callables reuse that exact pattern. Privileged writes already run server-side through the Admin SDK (`grantSchoolLicence` , the Stripe/RevenueCat webhooks), and `firestore.rules` denies all cross-user reads. Primary and, in v1, only user of this console.
- **Sales/ops (future role)** — a narrower `crm` claim for someone who works the pipeline but should not see, e.g., raw billing identifiers. Out of scope for v1; the data model leaves room for

it.

- **Customer** — the subject of a record, not a user of the console. Either a **pilot** (an existing `users/{uid}`) or, later, a **school** organization.

3. Scope

In scope (v1)

- A **customer list**: every pilot account with at-a-glance plan, status, last active and signup date; searchable by email/name; filterable by plan and status (trialing / active / past-due / canceled / free).
- A **customer detail** view that joins, read-only:
 - account basics from `users/{uid}` (display name, email, locale, created),
 - the plan from the `users/{uid}.entitlement` field (plan, status, term/expiry),
 - a billing summary (Stripe customer/subscription, last invoice) — read via a Cloud Function, never client-side Stripe keys,
 - the customer's support notes (below).
- **Internal notes**: staff can append timestamped notes to a customer (the one thing the CRM *writes*).
- A **B2B pipeline (leads)**: a lightweight kanban of flight-school prospects — stage, contact, value estimate, next action — feeding the "Fly GACA for Schools" engine in Phase 5.
- An **overview**: a few operational counters (active subs, trials ending in 7 days, past-due, open leads).

Out of scope (v1)

- **Editing billing**. Refunds, plan changes and cancellations stay in Stripe / the customer's own portal. The CRM links out; it does not mutate billing.
- **Editing the customer's account or entitlement** beyond notes. The `entitlement` field is written only by the billing webhooks (Stripe / RevenueCat) and `grantSchoolLicence` — the CRM must not become a second writer that drifts from billing.
- **Bulk email / marketing automation**. Transactional email stays in the billing flow; campaigns are a later, separate decision (Phase 8).
- **An external CRM integration** (HubSpot/Salesforce). Revisit only if the pipeline outgrows the in-house kanban.
- **Self-service org management** for school owners — that is the Phase 6 product surface, not this internal console.

4. Data model (Firestore)

Customer truth stays where it already lives — the account and its plan are both on `users/{uid}` (profile fields + the server-written `entitlement` field). The CRM adds only operator-owned, server-only collections.

```
crmNotes/{noteId}
  subjectType: 'user' | 'lead'
  subjectId: string           // uid or leadId
  authorUid: string           // staff who wrote it
  body: string
  createdAt: ts

leads/{leadId}                // B2B pipeline – flight schools / orgs
  orgName: string
  contactName: string
  contactEmail: string
  phone: string
  stage: 'new' | 'contacted' | 'demo' | 'proposal' | 'won' | 'lost'
  seatsEstimate: number
  valueEstimateSar: number    // currency in SAR, integer halalas optional
  ownerUid: string            // staff owner of the deal
  nextAction: string
  nextActionAt: ts
  source: string              // 'inbound' | 'referral' | 'event' | ...
  createdAt, updatedAt: ts

# Derived/read-only, NOT stored by the CRM:
#   users/{uid}                – account profile (owned by the user)
#   users/{uid}.entitlement     – plan/status/term (owned by billing webhooks)
#   Stripe / RevenueCat sub    – fetched live via a Cloud Function
```

No customer-facing collection changes. `crmNotes` and `leads` are server-only (written via the Admin SDK from CRM callables), like the existing `adelQuota` counters. A `crmIndex` summary doc (denormalized list rows) is an optional optimization for the customer list and is deferred until the list is slow.

5. Security model

- **Reuse the existing `admin` claim.** No new auth primitive is needed — `functions/school.js` already gates `grantSchoolLicence` / `revokeSchoolLicence` with `assertAdmin(req)` (`req.auth.token.admin === true`). Every CRM callable calls the same check first. Worth extracting `assertAdmin` into a shared helper (e.g. `functions/auth.js`) so `school.js` and `crm.js` share one definition.
- Every CRM read and write goes through an **admin-guarded Cloud Function**, which uses the Admin SDK and so **bypasses** `firestore.rules`. The console never reads another user's `users/{uid}` document directly from the client.
- `firestore.rules` stays as it is — strict per-user isolation with a deny-all `match {/document=**}` catch-all. `crmNotes` and `leads` fall under that catch-all and are therefore

unreadable/unwritable by any client, reachable only through the admin callables. No new client-facing rule is needed (and none should be added — these are operator data, not user data).

- **Billing identifiers** (Stripe customer/subscription IDs, card metadata) are fetched server-side and returned as a minimal summary; raw secrets and full payment data never reach the browser.
- **Auditability**: notes record `authorUid` + `createdAt`; lead stage changes stamp `updatedAt` and `ownerUid`. This is a PDPL-relevant surface — staff access to personal data should be attributable.
- The console is `noindex, nofollow` and sign-in gated, like the existing `admin.html` stub.

6. Surfaces

- `admin.html` — grows from the current stub into the console shell, or a new `crm.html` is added alongside it. (Decide in milestone 1; reuse the shared header/footer stamper either way — never hand-edit chrome.)
 - **Overview** — operational counters.
 - **Customers** — list + detail (account · entitlement · billing · notes).
 - **Pipeline** — leads kanban.
- `functions/crm.js` (new) — admin-only callable RPCs:
 - `crmListCustomers({ query, plan, status, cursor })`
 - `crmGetCustomer({ uid })` — joins account + entitlement + billing summary
 - `crmAddNote({ subjectType, subjectId, body })`
 - `crmListLeads()` / `crmUpsertLead(lead)` / `crmSetLeadStage({ leadId, stage })`
 - Exported from `functions/index.js` next to `createCheckoutSession`, `grantSchoolLicence` and the other callables.
- Reuses `functions/entitlements.js` and the existing billing modules (`functions/stripe.js`, `functions/revenuecatWebhook.js`) for the plan/billing summary, rather than duplicating that logic.

7. Open questions

- **Customer list at scale.** Firestore has no full-text search and no server join. Start by querying `users` on the `entitlement` plan field (the paying set is smallest and most interesting) and enriching per-row; revisit a `crmIndex` or external search if it gets slow.
- **Separate `crm.html` vs. growing `admin.html` ?** Leaning toward one admin console with tabs to avoid a second auth shell.
- **A narrower `crm` (sales) claim** distinct from `admin` — needed when a non-engineer works the pipeline. Defer until there's a second operator.

- **PDPL access logging.** Do we need to log *reads* of personal data, not just writes? Likely yes for paid/staff access — confirm with the LAWYER-BRIEF scope.
- **Leads ↔ accounts.** When a school lead converts (`grantSchoolLicence`), how do we link `leads/{leadId}` to the resulting school/cadets? Add a `linkedSchoolId` once the schools data model from `spec-instructor-dashboard.md` lands.

8. Milestones

1. **Console shell + auth.** Stand up the admin-gated `admin.html` / `crm.html` shell behind the existing `admin` claim (reusing / extracting `assertAdmin`). (No `firestore.rules` change — `crmNotes` / `leads` stay under the deny-all catch-all, reached only via callables.)
2. **Customers (read-only).** List + detail joining account, entitlement and the Stripe billing summary.
3. **Notes.** Append-only internal notes on a customer.
4. **Pipeline.** Leads kanban (stages, owner, next action) — the B2B on-ramp for "Fly GACA for Schools".
5. **Overview.** Operational counters (active / trialing / past-due / open leads).